deti
universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Software Architectures

VerdeMart

nopCommerce

Part 1: Architecture Checkpoint

Scenario C
Architectural Evolution of nopCommerce

Presented By: Carolina Reis (131193 · Tech Lead)
Francisco Arada (131117 · Modeler)
Guilherme Silva (131143 · Architect)

05 May 2026

What You'll Discover

01 Scenario C: Omnichannel Commerce Core

02 Five Architectural Pressure Points

03 Current State Analysis

04 Domain Model & Bounded Contexts

05 Quality Attribute Scenarios

06 Design Framework: Attribute-Driven Design (ADD)

07 Target Architecture

08 Evolution Roadmap

09 ADR-001: Asynchronous Messaging via RabbitMQ

10 ADR-002: Outbox Pattern

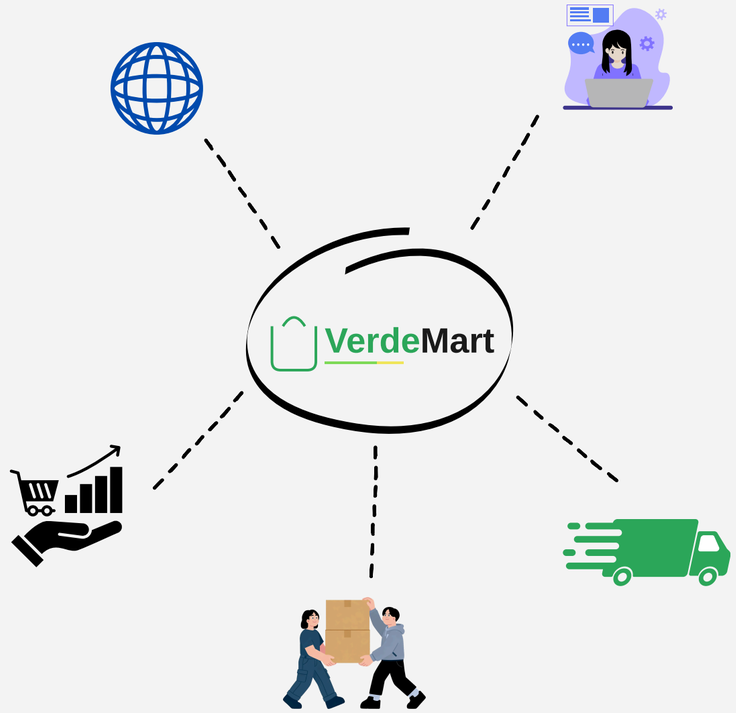
11 ADR-003: OrderSyncAdapter

12 Risk Plan & Validation Strategy

13 Feasibility Spike: RabbitMQ Integration Validated

14 What's Next: Part 2 Implementation

Scenario C: Omnichannel Commerce Core



VerdeMart Retail runs five channels:
web, Point of Sale, warehouse, shipping and support - sharing a customer base but not a data model.

✗ Item sold in-store ⇏ online stock updated
✗ WMS maintenance ⇒ checkout blocked
✗ Fulfillment state trapped inside ERP

Adding REST calls between systems would not fix this. It would create more places where a failed dependency propagates back into checkout.

Scenario C: Omnichannel Commerce Core

Strategic Goals

1

SG-1 Decouple order acceptance from operational system availability

2

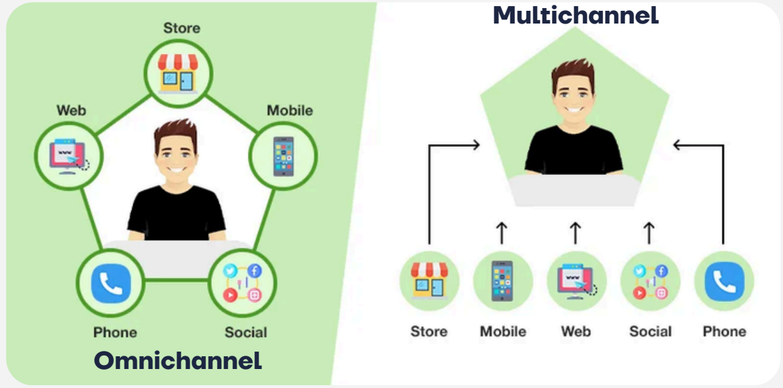
SG-2 Cross-channel visibility of stock, order state and fulfillment progress

3

SG-3 Remain useful during & after external system degradation

Why Scenario C?

The pressure point is **architecturally specific** and **directly demonstrable at runtime**: WMS goes down, storefront stays up.



Five Architectural Pressure Points

	Current Behaviour	Conflict with Goals
PP-1	Any external integration as synchronous call in req/resp cycle	Slow or unavailable dependency degrades checkout directly (SG-1)
PP-2	No built-in mechanism to publish order or stock events	External systems cannot react to state changes (SG-2)
PP-3	No inbound state-update contract from external systems	Cross-channel visibility requires an inbound contract (SG-2)
PP-4	No circuit-breaker, retry or fallback for external dependen-cies	Failures propagate instead of being contained (SG-3)
PP-5	No reconciliation mechanism after a dependency outage	Data remains inconsistent indefinitely after recovery (SG-3)

- Critical (PP-1, PP-4)
- High (PP-2, PP-3, PP-5)

PP-1 & PP-4 produce customer-visible failures;

addressed directly by ADR-001 and ADR-003.

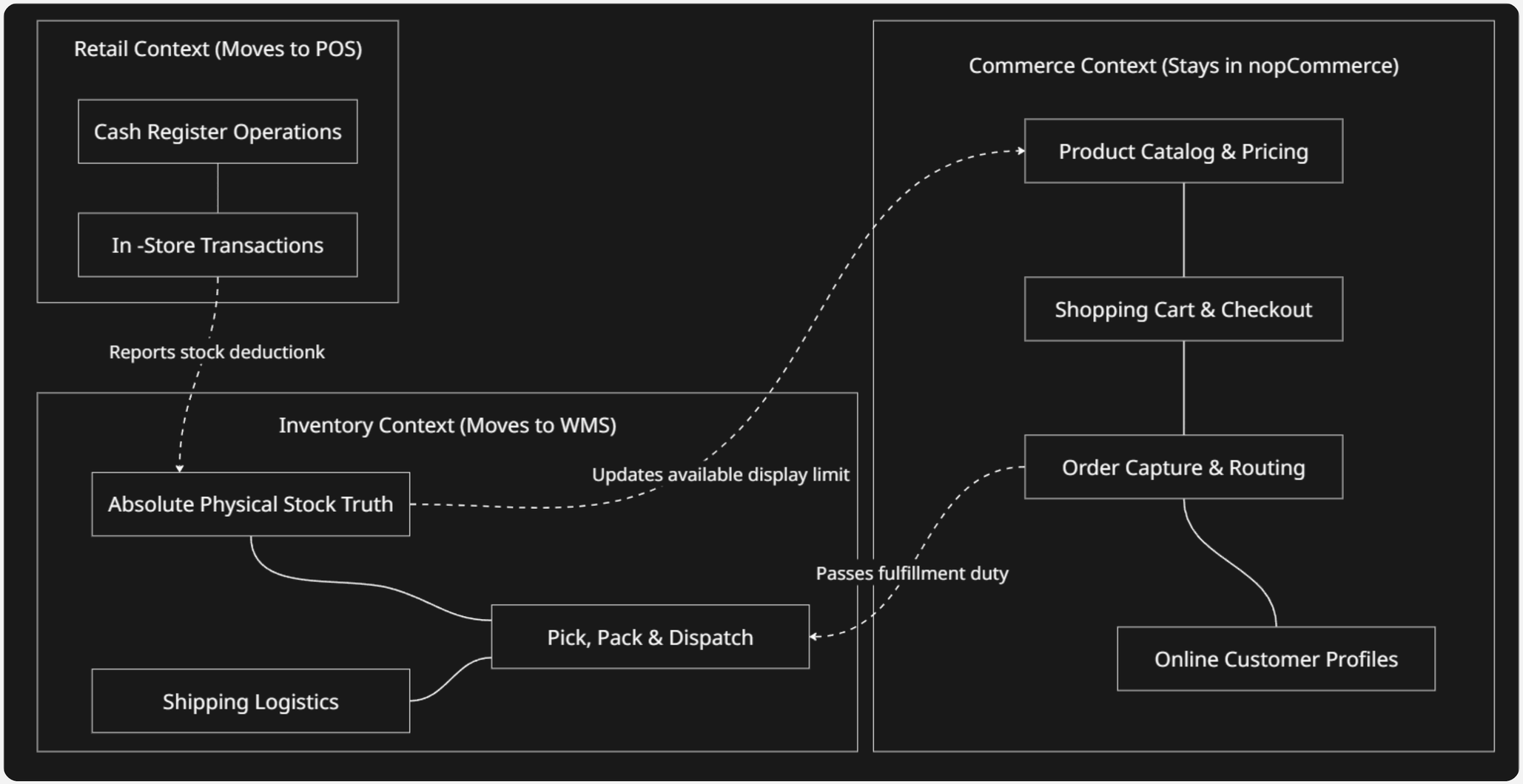
Current State Analysis

1. **Synchronous Coupling:** All operations execute synchronously against a single database, creating a bottleneck.

2. **No Native Async Events:** The current system is unable to safely broadcast "Order Placed" events to external operational systems.

3. **Fulfillment Rigidity:** If connected directly to a POS or WMS, the platform risks severe database collisions and cascading request timeouts.

Domain Model & Bounded Contexts



Quality Attribute Scenarios

ID	Quality Attribute	Stimulus → Response Measure	Drives
QAS-1	! Availability [Crit.]	Order placed while WMS/ERP unavailable → 100% accepted; sync ⩽2 min after recovery; zero orders lost	ADR-001, 002, 003
QAS-2	! Reliability [Crit.]	Broker restarts mid-publishing → zero message loss; reconnect ⩽10 s; duplicates handled idempotently	ADR-002
QAS-3	! Recoverability [High]	External system recovers → queue drained ⩽5 min; no duplicates; no manual restart required	ADR-003
QAS-4	i Consistency [High]	WMS stock update → visible in storefront ⩽30 s; staleness indicator served during WMS outage	Part 2
QAS-5	i Availability [High]	ERP slow/unresponsive → storefront ⩽500 ms; checkout success rate unaffected	ADR-001

Design Framework:
Attribute-Driven Design (ADD)

Rejected Alternative

ACDM (Architecture-Centric Design Method):
Centres on stakeholder workshops, not QA-driven structural design!
It misses the pressure problem.

Why ADD?

The core problem in Scenario C is not a missing feature. nopCommerce already places orders. What it cannot do is behave correctly under pressure.

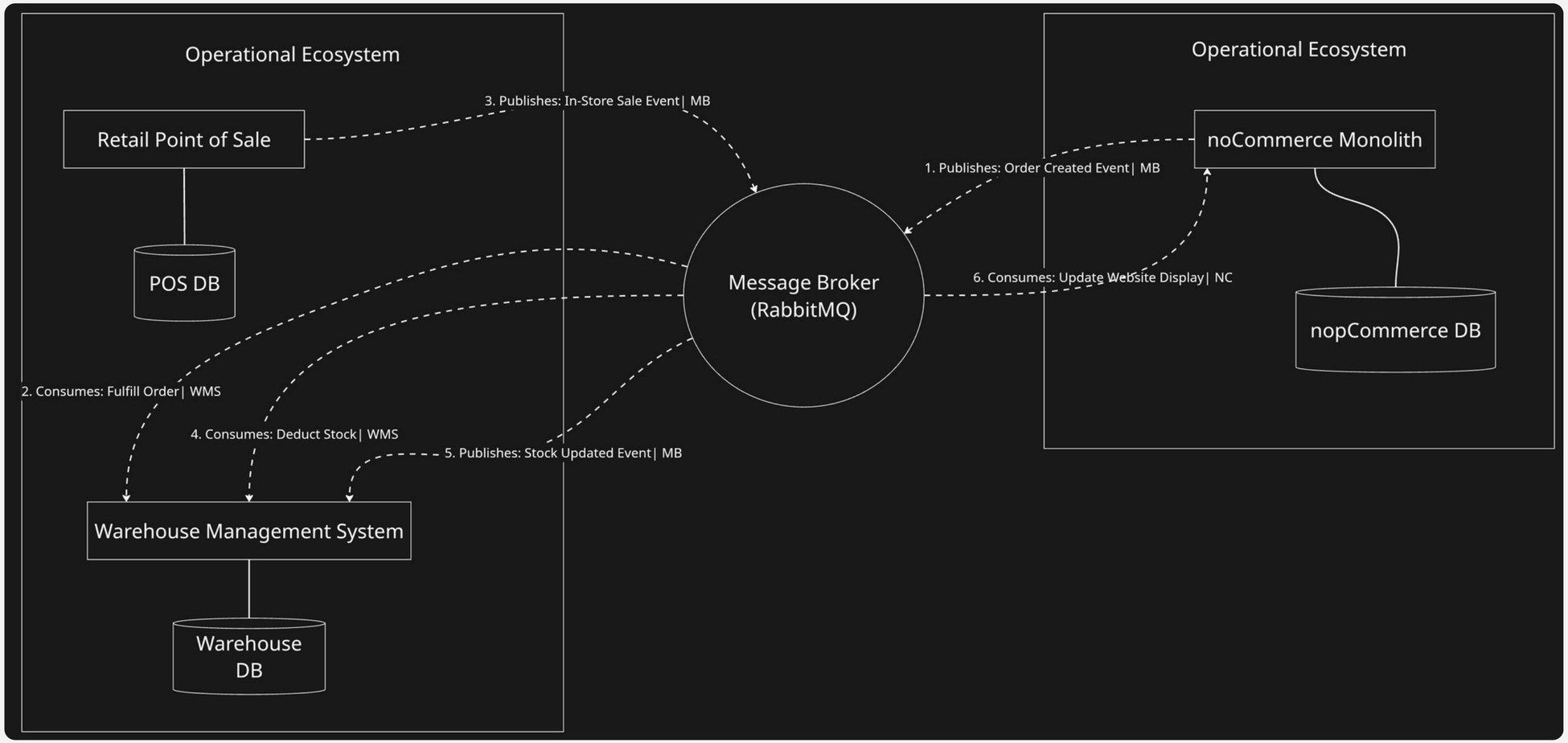
ADD forces an explicit answer: ***what does the system guarantee when things go wrong?***
Every structural decision is derived from a measurable QA scenario, not from a feature backlog.

ADD Traceability chain:

Scenario	Structural Decision	ADR
PP-1	QAS-1 (Availability)	001
PP-2	QAS-2 (Reliability)	002
PP-3	QAS-3 (Recoverability)	003
PP-4	QAS-4, QAS-5	—

Why RabbitMQ over direct HTTP?
QAS-1 requires checkout to succeed regardless of WMS state. That constraint rules out any synchronous dependency on the order path.

Target Architecture



Evolution Roadmap

P1: Infrastructure

Setup local nopCommerce, stand up RabbitMQ, and deploy WireMock simulators for WMS and POS.

P2: Async Flow

Intercept the checkout process and publish the "OrderCreatedEvent" to the RabbitMQ queue safely.

P3: Resilience Demo

Implement a kill switch to intentionally crash the WMS and test the core's recovery behavior.

P4: Final Defense

Compile the Evidence Pack, finalize the architecture report, and rehearse the live pressure demo.

ADR-001: Asynchronous Messaging via RabbitMQ

Decision:

nopCommerce publishes an **order.placed** event to a durable RabbitMQ queue immediately after persisting the order.

External systems (ERP/WMS) consume independently at their own pace.

nopCommerce's responsibility ends at publishing. The broker absorbs availability mismatches between producers and consumers.

Queues: durable=true
Messages: DeliveryMode=Persistent

Rejected Alternative 1

Synchronous REST: Call ERP API before returning checkout response.

✗ Makes checkout reliability bounded by ERP reliability → directly violates QAS-1. Every ERP outage becomes a checkout outage.

Rejected Alternative 2

Outbound Webhooks: nopCommerce POSTs to a list of subscriber URLs on order creation.

✗ Shifts retry and timeout management back into nopCommerce. message broker handles this as a first-class concern with durability and consumer acknowledgement built in.

✓ **Satisfies QAS-1 & QAS-5: checkout decoupled from ERP/WMS state.**

✓ **Foundation for ADR-002 (reliable publish) and ADR-003 (independent consumer)**

ADR-002: Outbox Pattern

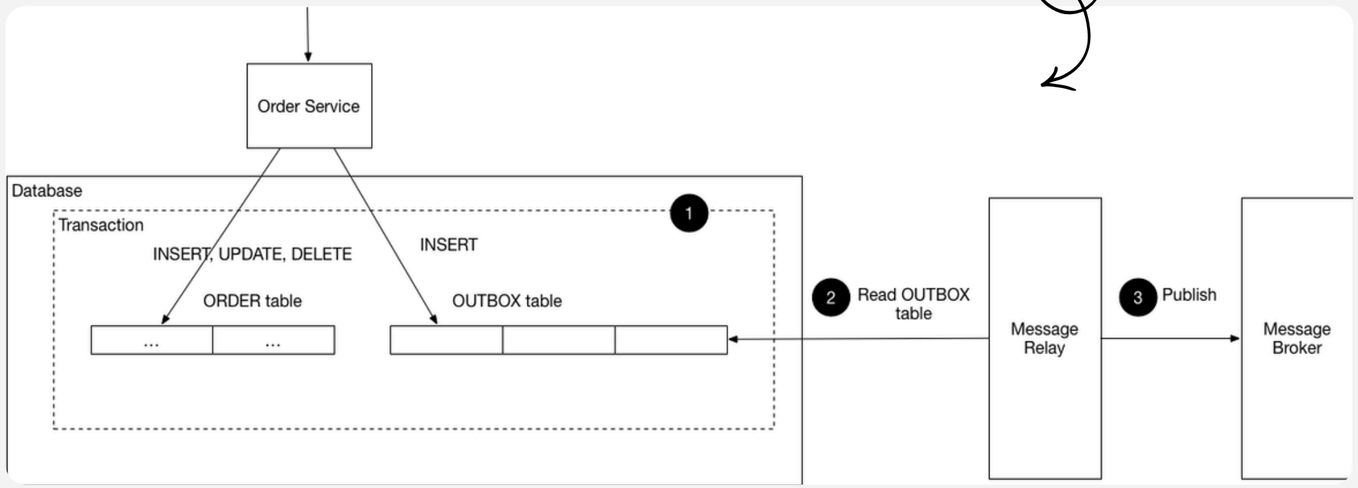
ADR-002: Outbox Pattern | QAS-1, QAS-2

✗ Problem: publishing directly to RabbitMQ inside SaveOrder() creates a dual-write. If the DB commits but the publish fails, the order is silently lost downstream.

✓ Decision: write an OutboxMessage record in the same DB transaction as the order. A background service polls, publishes with publisher confirms, and marks processed only after broker ack.

Rejected:
direct publish (no atomicity) · 2PC (RabbitMQ has no XA support) · Debezium (out of scope)

debezium



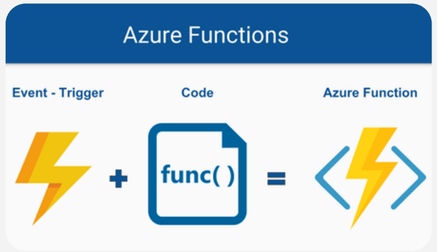
ADR-003: OrderSyncAdapter

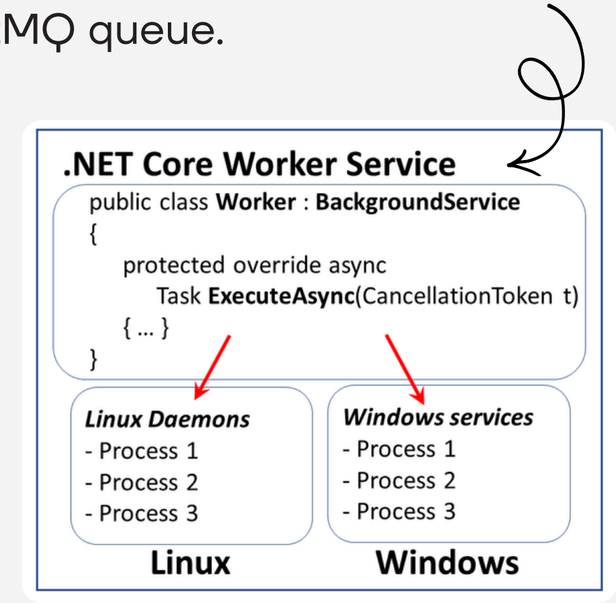
ADR-003: Extract OrderSyncAdapter | QAS-1, QAS-3

✗ Problem: a plugin running inside nopCommerce shares its process, DB connection pool and deployment lifecycle. No enforced boundary; impossible to restart or scale independently.

✓ Decision: extract as an independently deployable .NET Worker Service in its own Docker container. Zero reference to any Nop.*library. Only coupling to nopCommerce: the RabbitMQ queue.

Rejected:
inline plugin (no isolation) · Azure Function (cloud dependency, obscures the architectural boundary)





Risk Plan & Validation Strategy

ID	Risk	Prob	Impact	Status
R1	Message loss on broker restart (non-durable queue)	Med	Critical	✓ Mitigated (spike)
R2	Stock divergence during extended WMS outage	Med	High	Part 2
R3	Duplicate orders forwarded to ERP on message retry	Med	High	Part 2
R4	OrderSyncAdapter loses RabbitMQ connection silently	Med	High	✓ Mitigated (spike)
R5	Dead-lettered messages silently dropped	Low	High	Part 2
R6	Event schema change breaks OrderSyncAdapter	Low	High	Part 2

✓ **R1 & R4 validated before Part 1** by the feasibility spike: publish a persistent message, restart broker, confirm the message survives and the consumer reconnects automatically.

▲ **R2, R3, R5, R6** require Part 2 implementation: OrderSyncAdapter, Polly retry, WireMock fault injection and integration tests.

Feasibility Spike:

RabbitMQ Integration Validated

Goal:

Prove that .NET can connect to RabbitMQ, declare a durable queue, and publish a persistent message with publisher confirms before investing in the full plugin.

Validated:

✓ **R1:** durable=true + DeliveryMode=Persistent:
message survives broker restart

✓ **R4:** AutomaticRecoveryEnabled=true:
consumer reconnects within 10 s, transparently

Environment:

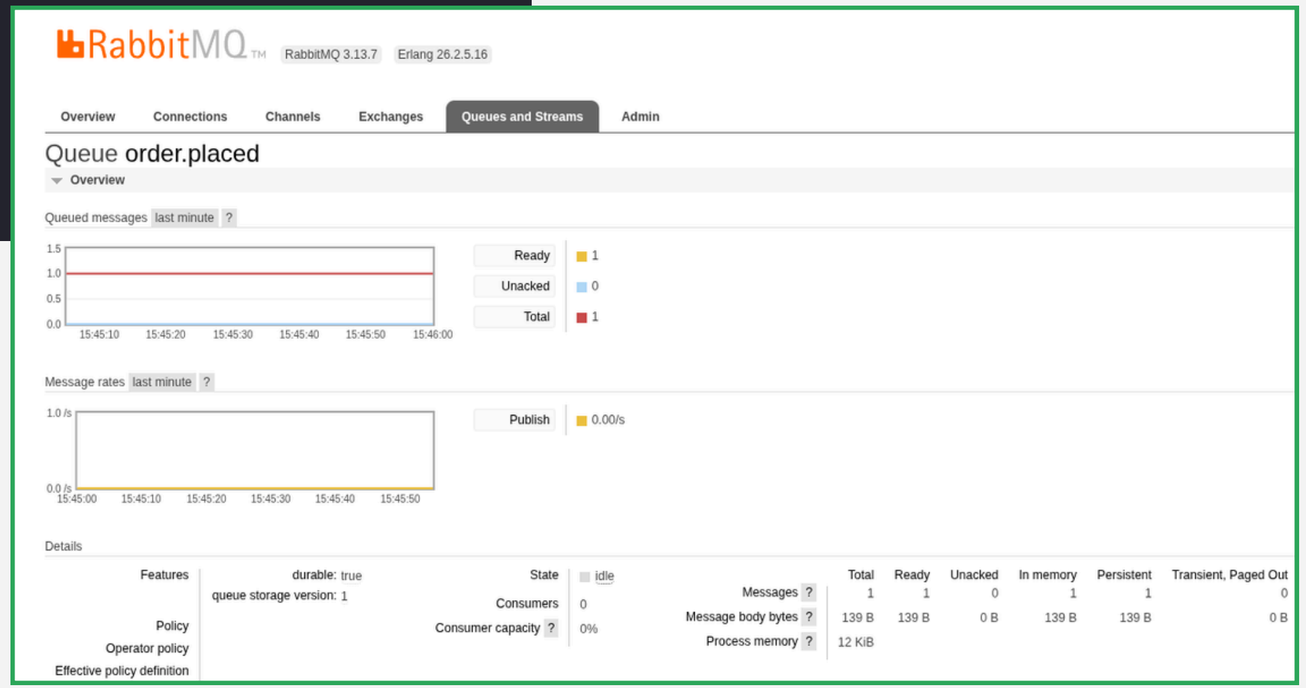
```
Kali Linux
.NET 9.0.312
RabbitMQ server 3.13.7 (Docker)
RabbitMQ.Client 6.8.1 (.NET)
```

Feasibility Spike:

RabbitMQ Integration Validated

```
┌──(meowstermind㉿LEGION-LULU)-[~/…/assignment2/nopCommerce-Omnichannel-Core/spike/rabbitmq-spike]
└─$ dotnet run
=== RabbitMQ Feasibility Spike ===
Connecting to localhost:5672...
Connection established.
Queue 'order.placed' declared (durable=true).
Publisher confirms enabled.

Publishing message: {"orderId":42,"customerId":7,"totalAmount":129.99,"currency":"EUR","timestamp":"2026-05-04T14:40:04
.0412881Z","source":"feasibility-spike"}

[OK] Broker acknowledged the message (publisher confirm received).
[OK] Message is durable and will survive a RabbitMQ restart.

Verify in the RabbitMQ management UI:
  http://localhost:15672  (guest / guest)
  Queue: order.placed  ->  Messages Ready: 1

=== Spike complete ===
```

Spike output: .NET connects to RabbitMQ, declares durable queue and receives publisher confirm



RabbitMQ Management UI - queue order.placed: durable, 1 message ready, 1 persistent

Feasibility Spike:

RabbitMQ Integration Validated

```
┌──(meowstermind㉿LEGION-LULU)-[~/…/AS/assignment2/nopCommerce-Omnichannel-Core/infrastructure]
└─$ docker restart verdemart-rabbitmq
verdemart-rabbitmq

┌──(meowstermind㉿LEGION-LULU)-[~/…/AS/assignment2/nopCommerce-Omnichannel-Core/infrastructure]
└─$ curl -s -u guest:guest http://localhost:15672/api/queues/%2F/order.placed \
  | python3 -c "import sys,json; d=json.load(sys.stdin); \
    print(f'messages_ready: {d[\"messages_ready\"]}  messages_persistent: {d[\"messages_persistent\"]}  durable: {d[\"durable\"]}')"
messages_ready: 1  messages_persistent: 1  durable: True
```

R1 validated: message survives broker restart - messages_ready: 1, durable: True

```
23    var factory = new ConnectionFactory {
24        HostName = Host,
25        Port = Port,
26        UserName = "guest",
27        Password = "guest",
28        // R5 mitigation: automatic recovery reconnects after broker restart
29        AutomaticRecoveryEnabled = true,
30        NetworkRecoveryInterval = TimeSpan.FromSeconds(5)
31    };
```

R4 mitigated: AutomaticRecoveryEnabled = true - connection restored transparently after restart

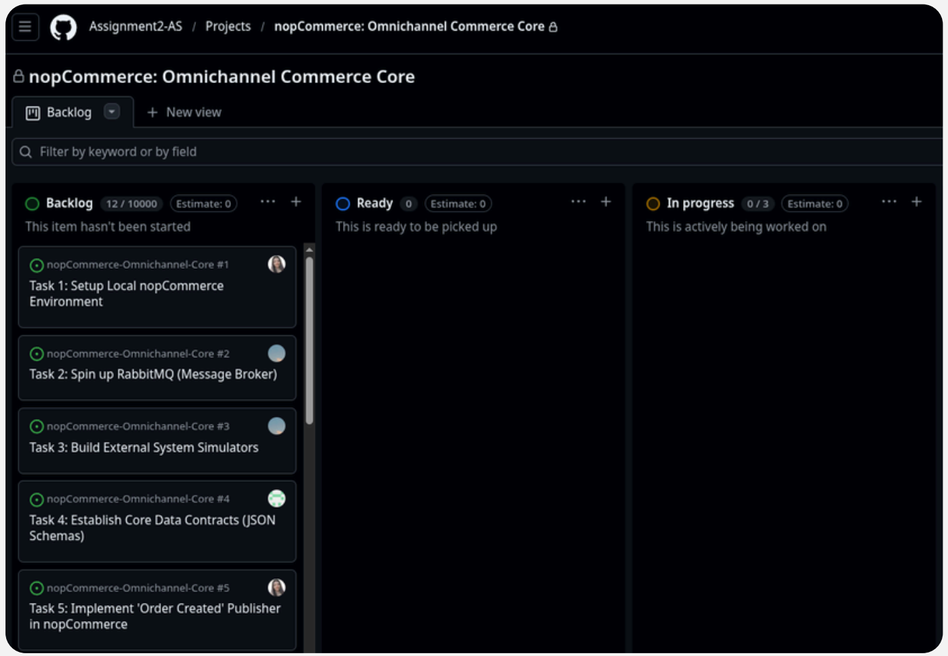
What's Next: Part 2 Implementation

➤ Phase 2 · May → June

- Polly retry + circuit breaker in OrderSyncAdapter
- Dead-letter queue + monitoring
- Stock visibility (QAS-4, QAS-5)
- End-to-end degradation & recovery demo

code Phase 1 · Now → May

- Nop.Plugin.Integration.OrderPublisher Outbox plugin inside nopCommerce
- OrderSyncAdapter .NET Worker Service consuming from RabbitMQ
- WireMock stubs for ERP and WMS with configurable fault injection



GitHub Project - Backlog

Part 1

References

[1] Khan, W. D. (2025). Omnichannel vs Multichannel Marketing: How Are They Different?. WPMaps.
https://wpmaps.com/blog/omnichannel-vs-multichannel-commerce/

[2] Richardson, C. Pattern: Transactional outbox. microservices.io.
https://microservices.io/patterns/data/transactional-outbox.html

[3] RabbitMQ. Consumer Acknowledgements and Publisher Confirms.
https://www.rabbitmq.com/docs/confirms

[4] Shahed C. (2020). Worker Service in .NET Core 3.1. Wake Up and Code!.
https://wakeupandcode.com/worker-service-in-net-core-3-1/

[5] Matos, L. (2024). Azure Functions: Unlocking the power of serverless computing.
https://luismts.com/azure-functions/